Data processing | technical handoff

Kolosok / BMW challenge - team briefing - 11 October 2026

What this part of the project does

This pipeline turns customer feedback workbooks into traceable topic candidates, deterministic statistics, evidence-linked English AI summaries, a readable local report and a versioned SQLite data library. It also imports sales volumes, customer studies and option-list PDFs. The original files and source references remain available for audit and for a later product integration.

Current state	Verified value
F70 feedback	4,076 source rows; 2,128 unique review IDs
F70 analysis	460 topics; 7,596 fragment occurrences; status complete
F70 assignments	6,906 distinct review-topic pairs; 2,115 reviews assigned to at least one topic
Other feedback	G60: 4,000 unique reviews; G70: 4,000 unique reviews; awaiting analysis
Supporting data	36 sales observations; 1,833 study observations; 78 PDF pages
Library	8 imported source files, 1 attached F70 analysis run, 54 passing Python tests

> Current state is a local working copy. The repository changes have not been committed or pushed because the required Entire checkpoint is unavailable in this task session.

Architecture at a glance

Input XLSX -> feedback_topics.py -> stage-1 topics JSON -> summarize_topics.py -> stage-2 summary JSON -> export_topics.py -> offline HTML + Markdown + JSONL + topic details. In parallel, data_library.py imports the XLSX/PDF sources into SQLite and attaches the two validated analysis JSON files to the exact workbook version. There are four explicit operations. Importing a workbook does not start AI analysis or refresh an old HTML report.

1. Inputs, identity and source preservation

Feedback workbooks need ID and Customer Feedback columns; by default the first worksheet is analyzed. --sheet, --id-column and --text-column can select another layout. Country, Engine Type, Derivate (e-code), Feedback Type and existing VFC labels are preserved as metadata. The clustering text input is only Customer Feedback, so existing classifications do not determine the new topics. English token and sentiment rules are used; no translation is performed.

Rows with the same ID and identical text become one review. All source Excel row numbers and differing metadata values are retained. Conflicting texts for one ID fail the stage-1 run. One review can contain several aspects and can later belong to several topics. Excel cached cell values are read; formulas are not recalculated.

The data library uses the lowercase filename stem as a dataset key unless --dataset is supplied. For an updated workbook, reuse the same key. A new byte sequence creates an immutable full-snapshot version; the same SHA-256, dataset key and importer version are a no-op. Re-importing an older version does not reactivate it. A failed import rolls back its file transaction. Current views select the highest successful version number for each dataset.

Input	Treatment in the library
F70, G60, G70 feedback XLSX	Raw bytes, sheets, nonempty rows, deduplicated reviews and source-row links
sales_volumes.xlsx	36 integer volumes by model group, market and year; scenario unspecified
Customer-studies XLSX	1,833 means or column shares by metric, model, region and year
Three OptionList PDFs	Original bytes plus extracted text on 78 numbered pages

The library retains original XLSX/PDF bytes as SHA-256-addressed assets. For spreadsheets, it also records sheet names, nonempty rows, source row numbers, cell data types, number formats and formulas. Numeric observations point to exact source cells. Unknown workbook layouts can be archived as generic raw rows, but require a dedicated adapter before they become meaningful analytic observations.

2. Stage 1: topic discovery and rule-based sentiment

The stage-1 implementation is feedback_topics.py. It is local and deterministic: NumPy and openpyxl are used, with no neural model, pretrained model, external API, SciPy or scikit-learn. Its output is F70_topics.json, schema version 1.0. The original workbook hash, sheet, parameters, rules, reviews, fragments, topic assignments and warnings are recorded in that JSON.

1. Split review text at sentences, line breaks and selected contrast clauses. Long clauses are divided into pieces of at most 65 words. Keep exact original substrings and start/end offsets. Whitespace-normalized duplicate fragments share one clustering observation, but every original occurrence remains available.
2. Lowercase and tokenize; remove English stopwords and apply editable word aliases. Build unigrams and adjacent-token bigrams. Only review text contributes to topic vectors; metadata is excluded.
3. Keep terms appearing in at least 2 unique fragments and in no more than 75% of them, capped at 10,000 features. Sublinear TF = 1 + ln(term count); smoothed IDF = 1 + ln((1 + unique fragments)/(1 + document frequency)). L2-normalize vectors.
4. Compute cosine similarity, then agglomerative average linkage with a nearest-neighbor-chain implementation. The distance cut is 0.94 by default. The exact similarity matrix takes O(n^2) memory and is limited by --max-memory-mb (default 768). Empty vectors remain as unassigned fragments, not silently dropped.
5. Name clusters from high-weight TF-IDF terms. Rank topics by distinct review count; assign T0001-style IDs. A topic counts a review once, even if several of its fragments match. Different fragments can place one review in multiple topics.

Sentiment is independent from clustering. sentiment_rules_en.json contains editable English phrase/word rules, scores, request patterns and a local negation window. Longer overlapping phrases win. Fragment labels are positive, negative, mixed or undetermined; neutral is reserved for future explicit annotations. A request is a separate flag and is not automatically negative. Review-topic sentiment combines the fragment labels within that topic. Rule matches and negation evidence remain in JSON.

F70 stage-1 result	Value
Vocabulary; unique normalized fragments	4,111 terms; 7,388 unique fragments
All original fragment occurrences	7,596
Assigned / fully unassigned reviews	2,115 / 13
Unassigned fragments	525, retained in the source JSON
Small topics below 3 reviews	15, flagged rather than removed

3. Stage 2: exact statistics and AI explanations

summarize_topics.py validates all review, topic and fragment references before calculation. It emits F70_topics_summary.json, schema version 2.0. Counts, shares, segment comparisons, sentiment assignments and flags are computed in Python. AI cannot change those values. Global sentiment counts each original review once across all its fragments, including content outside any topic. Topic counts can overlap, so their sum is not the corpus total.

For each topic, the denominator of a model/country/engine share is that topic's review_count. For a segment's negative_or_mixed_rate, the numerator is negative + mixed reviews in the segment and topic, and the denominator is all reviews in that segment and topic. topic_prevalence_in_source_segment divides matching topic reviews by all source reviews in that segment. Country-by-engine counts are computed from joint observations, never from a product of marginal shares. Fractions run from 0 to 1.

Descriptive signals require a segment of at least 10 reviews by default. A signal is emitted for a negative-or-mixed rate of at least 0.8, or a gap of at least 0.2 versus the rest of the breakdown when the comparison segment also has at least 10 reviews. These are exploratory flags: there is no significance test, multiple-testing correction or causal claim.

For AI prose, the script selects up to 16 example fragments per topic, covering available sentiments, flagged segments and lexical centrality. This is a deliberately stratified sample, not a random prevalence sample. The OpenAI Responses API receives selected excerpts and computed statistics to write English titles, concise topic/sentiment summaries, a corpus summary and evidence-linked notes. Structured JSON output is required. Returned fragment and signal IDs, sentiment alignment, schema and numeric prose are checked; invalid responses are retried or rejected. Source text is treated as untrusted data. The API key is read from the environment or hidden input, never written to the report; the request uses store=false. API calls incur cost.

The default model in the current script is gpt-5.4, with five concurrent workers. A cache keyed by model, prompt, schema and payload supports resuming. --stats-only performs the entire numeric stage offline; --limit N caps new summaries in one run. status is statistics_only, partial or complete. For F70, the stored status is complete: all 460 topics and the corpus summary are present. The summary retains method definitions, settings, source checksum, per-topic evidence and limitations.

4. Stage 3: readable and machine-readable exports

export_topics.py reads an existing stage-2 JSON. It makes no AI calls and does not recalculate labels. For F70, the default output directory is F70_topics_summary.readable. Its 465 files are 460 full topic JSON records plus five top-level files: report.html, overview.md, overview.json, topics.jsonl and README.md.

File	Intended use
report.html	Self-contained offline browser report: search, filters, sorting, sentiment and country breakdowns, quotes, evidence and limitations
overview.md	Short human overview and leading topic rankings
overview.json	Exact global values, settings, definitions, warnings and provenance
topics.jsonl	Compact, one-topic-per-line AI catalog; links to complete topic detail files
topics/Txxxx.json	Full original stage-2 record for a selected topic
README.md	Reading order, denominators, overlap warnings and retrieval guidance

Read README.md, overview.md and overview.json first; scan topics.jsonl to choose topic IDs; load detail JSON only for selected topics. This avoids sending the entire summary to an AI context. The F70 JSONL catalog is 622,557 bytes versus 7,010,754 bytes for the summary, about 11.3 times smaller. Compactness does not replace the full evidence. Quotes remain verbatim and are escaped before HTML rendering; browser labels and generated prose are English.

The HTML report can be opened directly in a normal browser. report_template.html is only a template, not the finished report. Re-running export_topics.py regenerates the selected output directory. --top limits only the Markdown ranking; --small-sample changes only the display marker. All topics remain in HTML, JSONL and detail files.

5. Versioned SQLite library and portable handoff

data_library.py, library_schema.sql and DATA_LIBRARY.md define the storage boundary. The default database is data_library/library.sqlite. The schema is version 1. The current database has 8 source datasets and 1 attached F70 analysis run. Every analysis run references the exact feedback workbook version and both original analysis JSON hashes; topic IDs such as T0001 are scoped by run. Deterministic UUIDs namespace review identities by dataset and observations by version/source cell.

Tables / views	Purpose
assets, datasets, dataset_versions	Original bytes, logical identities and immutable snapshots
sheets, source_rows	Excel layout, typed raw rows, cell formats and formulas
reviews, review_rows	Unique feedback IDs with all supporting source rows
sales_observations, study_observations	Typed numeric facts linked to source cells
document_pages, import_issues	Page text, OCR flags and interpretation warnings
analysis_runs, analysis_topics, topic_reviews, fragments	Version-bound topic analysis and evidence links
current_versions, current_reviews, current_sales, current_studies	Latest successful version of each source dataset

The importer checks duplicate IDs, conflicts, finite numeric values, source references and foreign keys. The check command runs SQLite integrity and foreign-key checks and verifies each asset's SHA-256. Ingest-all processes each input file in its own transaction and attaches matching *_topics.json / *_topics_summary.json pairs. Status lists F70 as Analysis ready and G60/G70 as Awaiting analysis. A new source version is pending until an analysis for those exact bytes is attached.

Export writes to an empty destination: a consistent SQLite backup, table JSONL files, original assets, schema.sql, schema.json, a readable STATUS.md and a manifest.json with SHA-256 and byte counts. All historical versions are included; current_* views identify active snapshots. The initial portable archive is data_library/exports/initial.zip. A target Python project can open library.sqlite directly. Other databases can load the JSONL under equivalent constraints, preserving dataset/version/run composite identities.

6. Operational recipe for the team

Run these commands from feedback_topics_package after installing Python 3.10+ dependencies from requirements.txt. Generate a new unique prefix for each feedback dataset. Keep the workbook and generated JSON files together so ingest-all can attach the matching pair.

```
python -m pip install -r requirements.txt
python feedback_topics.py "G60_feedback_hackathon.xlsx" --output G60_topics.json
python summarize_topics.py G60_topics.json --output G60_topics_summary.json
python export_topics.py G60_topics_summary.json
python data_library.py ingest-all .
python data_library.py status
python data_library.py check
python data_library.py export --output data_library/exports/next-release
```

To inspect numeric results without API calls, replace the summarize command with --stats-only. To resume after a partial AI run, use the same output, model and cache. Do not run two summarizers against one output file. An imported workbook can be replaced by a full updated snapshot with python data_library.py ingest "updated.xlsx" --dataset g60_feedback_hackathon. The existing analysis of the older bytes does not make the new version ready. A new export directory is required after changes; an earlier archive or readable report is a snapshot.

Interpretation and integration limits

Clusters are lexical candidates and require a product manager's review; 0.94 is a chosen distance cut, not a probability or proof of 460 true topics. The English rule lexicon has no measured accuracy on an independent labeled sample. Sarcasm, distant negation, mixed aspects and language differences can be misclassified. Rare terms filtered by min_df can remain unassigned. Cosine similarity and rule scores are not confidence probabilities.

Sales sheets keep combined model groups such as G60_G68 and scenario=unspecified because the workbook does not label actuals versus forecasts. Customer-study means have unknown scale bounds and sample sizes; US shares are fractions, so Sample total = 1 represents 100%, not one respondent. Do not average incompatible measures. PDFs are indexed by page text; price/option/table relationships are not structured or validated. Pages without extractable text are flagged needs_ocr. Saved Excel formula caches may be stale.

Customer quotes and source fields remain in their original language. UI text, generated AI summaries and source-code messages are English. Source data, API secrets, database snapshots and exported bundles are excluded from Git. Share the bundle through approved team storage and validate manifest hashes after transfer.

Source map

Implementation: feedback_topics.py, sentiment_rules_en.json, summarize_topics.py, export_topics.py, report_template.html, data_library.py and library_schema.sql. Operating guides: README.md and DATA_LIBRARY.md in feedback_topics_package. Validation: test_feedback_topics.py, test_summarize_topics.py, test_export_topics.py, test_data_library.py and test_report_ui.js. Verified on the supplied F70 summary and current SQLite status; all 54 Python tests pass. The UI smoke test checks behavior with a document stub, not visual rendering in a browser.